

Lecture 11

Geometry-Aware ML II

— Manifolds & Diffusion Models

AI for Geometry — MM845

Edward Hirst

Instituto de Matemática, Estatística e Computação Científica
Universidade Estadual de Campinas

`ehirst@unicamp.br`

Setembro 2026

Overview

1. Building Architectures for Manifolds
2. Generative Modelling & Normalising Flows
3. Diffusion: Noise, Score, Training, Sampling
4. Generating Geometry
5. Summary

Recap (Lecture 10): Equivariance and Invariance can be built directly into network architectures; GNNs specialise to graph data.

Guiding question: *How can architectures represent manifolds?*
How can structure be generated from noise?

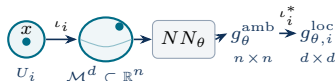
Building Manifold Architectures: Three Ways

Representing manifolds

A network takes numerical arrays as input and output. How can a manifold $\mathcal{M}$ be encoded as architecture to learn: functions, metrics, forms, or other geometric objects on it?

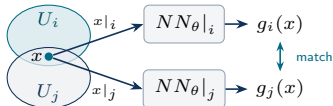
- **Extrinsic: Embedding Architectures**

Feed ambient coordinates $x \in \mathcal{M}^d \subset \mathbb{R}^n$ to the architecture. Restrict functions to $\mathcal{M}^d$; pull back ambient tensors using the embedding's Jacobian.



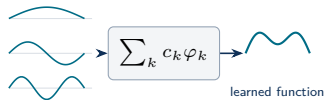
- **Intrinsic: Atlas Architectures**

Train in parallel the local coordinates on charts U_i . Match predictions on overlaps, using the correct transition rule for functions or tensors.



- **Spectral: Geometric Basis Features**

Choose Laplacian eigenfunctions φ_k and learn coefficients in $u_\theta(x) = \sum_{k=1}^K c_k(\theta) \varphi_k(x)$. Basis captures geometry; coefficients select the function.

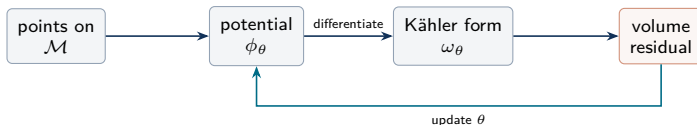


Case Study: Neural Calabi–Yau Metrics

A geometric object we know exists, but want to compute

Yau's theorem guarantees a unique Ricci-flat Kähler metric in each Kähler class on a compact CY manifold. Neural networks approximate metrics on examples such as the quintic threefold.

- **Represent:** sample points on the manifold using embeddings or charts. A network learns a potential correction ϕ_θ to a reference Kähler form ω_0 .
- **Constrain:** set $\omega_\theta = \omega_0 + i\partial\bar{\partial}\phi_\theta$; ensures global consistency and positivity $\omega_\theta > 0$. Known symmetries can be built in (L10).
- **Train:** minimise a Monge–Ampère residual, comparing ω_θ^n with the target volume form determined by the holomorphic volume form.



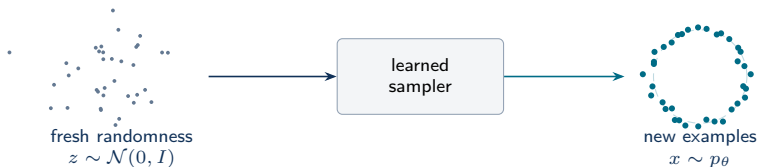
Why this is our first example

- These were the first examples of metric construction with AI (motivated by string theory).
- These **physics-informed neural networks (PINNs)** are the topic of Lecture 12.

Generative Modelling: Learning to Sample

A new task: learn a probability distribution

Given examples $x_1, \dots, x_N \sim p_{\text{data}}$, learn a model that produces *new samples* with similar statistical structure. There is no target label: the examples themselves are the training data.



- **Geometric viewpoint:** a complicated distribution may concentrate near a low-dimensional manifold — e.g. points near a circle in $\mathbb{R}^2$.
- **Normalising flows:** learn an invertible deformation of a simple distribution using change of variables.
- **Diffusion models:** gradually corrupt examples with noise, then learn many small steps that reverse this change in distribution.

Other Generative Families

- VAEs: learn a probabilistic encoder–decoder with a simple latent distribution.
- GANs: train a generator against a discriminator that separates generated from real samples.

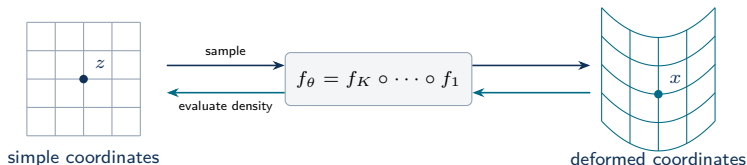
Warm-Up: Normalising Flows

Generation by an invertible change of variables

Choose a simple density p_Z , then learn an invertible differentiable map:

$$z \sim p_Z = \mathcal{N}(0, I) \xrightarrow{f_\theta} x \sim p_\theta(x) \approx p_{\text{data}}(x),$$
$$p_\theta(x) = p_Z\left(f_\theta^{-1}(x)\right) \left| \det \left[\frac{\partial (f_\theta^{-1})_j}{\partial x_k}(x) \right]_{j,k=1}^n \right|.$$

The Jacobian determinant corrects for local expansion or compression of volume.



- **Train:** minimise $-\frac{1}{N} \sum_i \log p_\theta(x_i)$, the negative log-likelihood. Compose layers with cheap inverses and Jacobian determinants.
- **Trade-off:** densities are tractable, but invertibility constrains the architecture. A diffeomorphism of $\mathbb{R}^n$ cannot put a full-dimensional Gaussian exactly on a lower-dimensional manifold.
- **Next idea:** learn to undo small amounts of noise, without requiring each network layer to be invertible.

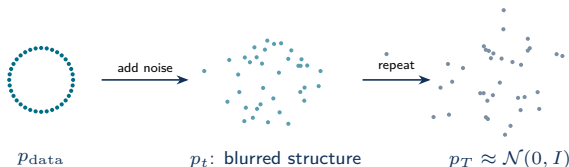
Diffusion I: Gradually Add Noise

The forward process is fixed — nothing to learn

Start with a data point x_0 . For noise levels $t = 1, \dots, T$, choose small variances $0 < \beta_t < 1$ and repeat

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, I) \text{ independently.}$$

Each step shrinks the signal a little and adds fresh Gaussian noise.



A shortcut: jump directly to any noise level

$$x_t = \alpha_t x_0 + \sigma_t \varepsilon, \quad \alpha_t = \prod_{j=1}^t \sqrt{1 - \beta_j}, \quad \sigma_t^2 = 1 - \alpha_t^2, \quad \varepsilon \sim \mathcal{N}(0, I).$$

This samples the same conditional distribution as t successive steps.

- Choose the schedule so $\alpha_T \approx 0$, making p_T approximately Gaussian.

Diffusion II: Which Way Leads Back to Data?

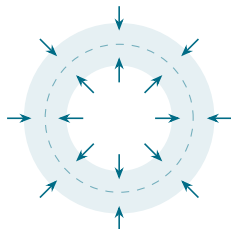
The score: a vector field for each noise level

Let $p_t(x)$ be the density after noising. Its **score** is

$$s(x, t) = \nabla_x \log p_t(x) = \frac{\nabla_x p_t(x)}{p_t(x)}.$$

It points towards the steepest local increase in log-density.

- At low noise, probability concentrates in a thin region near the data manifold; at high noise, its structure is much smoother.
- The log measures *relative* density changes. An unknown normalising constant disappears: $\nabla \log(\tilde{p}_t/Z_t) = \nabla \log \tilde{p}_t$.
- The score is not generally a nearest-point projection: it also reflects how probability varies *along* the manifold.
- We only have samples, so p_t and its score are unknown. The next slide turns score estimation into a regression task.



Low-noise schematic:
arrows point towards a high-density band.

Diffusion III: Learn by Denoising

Make a supervised task from unlabelled data

Draw x_0 from the dataset, choose t uniformly from $\{1, \dots, T\}$, and draw $\varepsilon \sim \mathcal{N}(0, I)$. Form $x_t = \alpha_t x_0 + \sigma_t \varepsilon$. Train a network $\varepsilon_\theta(x_t, t)$ to predict the added noise:

$$L(\theta) = \mathbb{E}_{x_0, t, \varepsilon} \left[\|\varepsilon - \varepsilon_\theta(\alpha_t x_0 + \sigma_t \varepsilon, t)\|^2 \right].$$



The noise target is known because we generated it ourselves.

Why predicting noise learns the score

Gaussian corruption has a known conditional density. Its log-gradient exposes the added noise:

$$\begin{aligned} q(x_t | x_0) &= \mathcal{N}(\alpha_t x_0, \sigma_t^2 I), \\ \nabla_{x_t} \log q(x_t | x_0) &= -\frac{x_t - \alpha_t x_0}{\sigma_t^2} = -\frac{\varepsilon}{\sigma_t}, \\ \varepsilon_\theta(x_t, t) \approx \mathbb{E}[\varepsilon | x_t, t] \quad \implies \quad s_\theta(x_t, t) &:= -\frac{\varepsilon_\theta(x_t, t)}{\sigma_t} \approx \nabla_{x_t} \log p_t(x_t). \end{aligned}$$

- The same network handles all noise levels; t tells it how corrupted the input is. Training uses ordinary batches and gradient descent (L3–4).

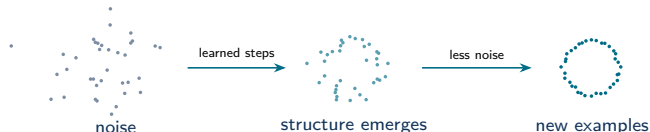
Diffusion IV: Generate by Reversing the Chain

After training, hold θ fixed and generate

Draw $x_T \sim \mathcal{N}(0, I)$. For $t = T, \dots, 1$, use this Gaussian reverse approximation:

$$x_{t-1} = \underbrace{\frac{x_t + \beta_t s_\theta(x_t, t)}{\sqrt{1 - \beta_t}}}_{\text{learned denoising mean}} + \underbrace{\sqrt{\beta_t} z_t}_{\text{fresh noise}}, \quad z_t \sim \mathcal{N}(0, I).$$

Use $\tilde{\beta}_t = \beta_t \sigma_{t-1}^2 / \sigma_t^2$, with $\sigma_0 = 0$: no noise at the final step. Return x_0 .



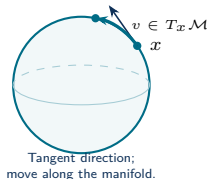
- **Distributional reversal:** generate new examples from the learned distribution; this does not undo a particular training example's noise.
- **Cost:** many network evaluations per sample; score error and step size limit accuracy.
- **Link to flows:** a continuous-time *probability-flow ODE* shares the diffusion's time-indexed distributions when the score is exact.

Diffusion on Manifolds: Generating Geometry

When the manifold is known, respect it throughout

For angles on tori, directions on S^2 , or rotations in $SO(3)$, use diffusion *on* $(\mathcal{M}, g)$. Brownian motion provides forward noising; a learned tangent score $\nabla_g \log p_t$ guides the reverse process. Here p_t is a density relative to Riemannian volume.

- On a connected compact manifold without boundary, Brownian motion approaches uniform volume. Heat kernels replace Gaussian transition densities.
- Geometric updates:** map tangent steps back to $\mathcal{M}$, e.g. with the exponential map. Ordinary vector addition can leave the manifold.



From samples to useful geometric candidates

- Condition $s_\theta(x, t, c)$ on a desired property c to steer generation.
- Treat each output as a candidate, verify geometric constraints and invariants independently.

Example Applications

Parametric PDEs (arXiv:2305.14703): diffusion models distributions over PDE solutions.

Evolving densities (arXiv:2405.02810): normalising flows solve Liouville density equations.

Knotted curves (arXiv:2501.12780): conditional diffusion generates a chosen knot type.

Summary

Key points

- Encode manifolds through embeddings, charts or spectral bases; neural CY metrics show how these choices support geometric computation.
- Generative modelling learns to sample a distribution from examples. Normalising flows do this through invertible changes of variables.
- Diffusion starts with a fixed noising process that gradually removes the data's structure.
- Denoising regression learns the score (a vector field at each noise level) which guides many small reverse noise sampling steps.
- Known manifold geometry can be built into diffusion; conditional generation steers candidates towards desired properties, followed by independent verification.

Next

Tutorial 11: Explore noising and train a toy diffusion model on circle/torus data.

Lecture 12: Physics-informed neural networks
...training geometric functions using PDE residuals.